Failure Synthesis System — Crash-Tolerant Failure Reconstruction for Autonomous Systems

---

Minor Project

Team Details:

Arnav Sachdeva	24103068	CSE	3rd Year
Kritansh Uppal	24103042	CSE	3rd Year
Sumit Rai	24103108	CSE	3rd Year
Vibhanshu	24103124	CSE	3rd Year

---

1. Introduction

Modern autonomous and distributed systems such as autonomous vehicles, drones, mobile robots, and multi-agent robotic systems continuously generate large amounts of telemetry and operational data. This data may include sensor observations, system states, planner decisions, fault events, task transitions, and other events generated during system operation.

When such a system fails, understanding the reason for the failure is often difficult. Many failures are non-reproducible: a system may encounter a particular combination of sensor conditions, internal states, and interactions with other agents only once. By the time an engineer investigates the failure, the original system state may no longer exist, and manually searching through large amounts of historical telemetry can be difficult.

The proposed project, **Failure Synthesis System**, is designed to address this problem by maintaining a crash-tolerant record of system events and providing mechanisms to retrieve the relevant evidence surrounding a failure. The retrieved evidence is then combined with domain-specific knowledge and an AI-based reasoning layer to produce a human-readable synthesis of what most likely happened.

The system is intended not simply as a logging mechanism, but as a complete pipeline from event collection and durable storage to contextual retrieval and failure explanation.

---

2. Problem Statement

Consider a fleet of autonomous drones operating together. Each drone continuously produces heterogeneous events such as position updates, sensor states, planner transitions, warnings, and faults, which are sent to a central system for recording and analysis.

When a failure occurs, the event that explains it may not originate from the affected drone itself. For example, suppose Drone 474 unexpectedly leaves its formation. Drone 298 may have previously experienced a sensor failure that altered its behavior, causing Drone 474 to react through its avoidance or planning system. Retrieving only Drone 474's events would therefore provide an incomplete view of the failure.

The system must consequently reconstruct relevant historical context across multiple sources and event types. However, continuously collecting events from a large number of sources introduces a second challenge: the volume of historical data makes unrestricted retrieval impractical. The system must identify and retrieve relevant evidence within bounded time ranges while preserving the relationships between events from different sources.

This becomes more difficult when the central event-processing system itself crashes. Events that have already been received but exist only in memory, or are only partially persisted when the process terminates, may be lost or left in an inconsistent state. The storage layer must therefore preserve recoverable event evidence and detect incomplete persisted data after an unexpected shutdown.

The project addresses these problems as a single pipeline:

Ingest Heterogeneous Events → Persist Them Safely → Retrieve Bounded Cross-Source Context → Reconstruct Relevant Evidence → Interpret the Evidence.

The final AI reasoning layer operates on this reconstructed evidence rather than directly searching an uncontrolled stream of raw telemetry. It uses deterministic decoding and available domain knowledge to interpret event payloads, while explicitly retaining unknown or uninterpretable information instead of assuming its meaning.

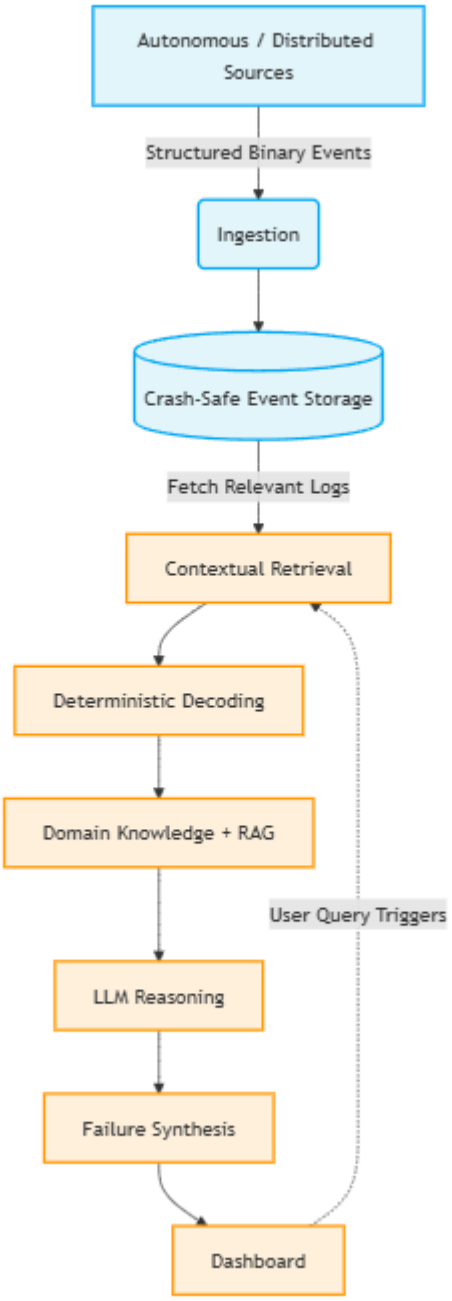
The resulting system is intended to provide a reliable historical evidence layer for failure investigation: even when the central system crashes, previously received events should remain recoverable, and when a failure occurs, the system should be able to assemble the relevant context needed to explain it.

---

3. Proposed System

The proposed system consists of a central event-processing system that receives structured binary events from autonomous or distributed sources.

A high-level representation of the system is:



The central system preserves the original event data while maintaining enough metadata to locate relevant events efficiently. A user can submit a failure-related query through a dashboard, such as:

> “Why did Drone 474 leave formation at approximately 10:07?”

The system then retrieves events from the relevant time period and sources, decodes the supported event types, and provides the resulting evidence to the reasoning layer. The final result is presented to the user through the dashboard along with supporting evidence and information about any data that could not be interpreted.

---

4. Event Ingestion

The system receives events from multiple producers. These producers may represent drones, robots, autonomous vehicles, or other distributed software components.

Events are represented using a structured binary format. The exact binary layout can be configured for a particular deployment rather than assuming that every possible system uses an identical protocol.

The central system requires a small set of metadata fields, such as:

- Source or agent identifier
- Event timestamp
- Event type
- Payload length
- Payload data

The byte offsets and representation of these fields are specified through configuration. This allows the ingestion system to work with different structured binary event formats without requiring the storage layer to understand the semantics of every possible event.

The timestamp generated by the producer is preserved as part of the event. The system does not assume that timestamps from different autonomous agents form a perfectly synchronized global clock. They are treated as temporal evidence associated with the source that generated them.

The payload itself can vary according to the event type. For example, one event may contain a sensor measurement while another may contain a planner state or fault code.

---

5. Crash-Tolerant Event Storage

A major part of the system is the crash-safe storage layer.

The event history is divided into **time-based segments**. For example:

10:00 – 10:05 Segment 1
10:05 – 10:10 Segment 2
10:10 – 10:15 Segment 3

This organization provides a natural first level of filtering during retrieval. If an investigation concerns events around 10:07, the system does not need to inspect the entire event history.

Each segment is further divided into fixed-capacity storage blocks. Blocks are allocated to individual sources as required rather than reserving a fixed amount of storage for every source.

For example:

Segment: 10:05 – 10:10

Drone 474 → B1 → B7 → B13
Drone 298 → B2 → B5 → B9
Drone 901 → B3 → B4

An individual block contains only complete events belonging to its associated source. An event is never split across two blocks. If the next event does not fit in the remaining capacity of a block, the remaining space is left unused and the event is written to the next block.

Each block therefore maintains a valid-data length indicating how much of its physical capacity contains actual events.

This design makes block scanning straightforward and also simplifies recovery after an unexpected crash.

Crash Recovery

The system distinguishes between an active segment and sealed segments.

10:00 – 10:05 SEALED
10:05 – 10:10 SEALED
10:10 – 10:15 ACTIVE

Only the active segment can contain a partially written tail after a crash. On recovery, the system scans the valid event records in the active storage region. Complete events are retained, while an incomplete final record is discarded or truncated.

For example:

Event A ✓
Event B ✓
Event C ✓
Event D partially written
↓
CRASH
↓
Recovery
↓
A, B, C retained
D incomplete → discarded

Once an event has crossed the system's durability boundary, a crash of the central event engine must not cause that event to disappear or become corrupted. The system can therefore restart and continue accepting new events while retaining the previously durable incident evidence.

The project focuses on protection against crashes or restarts of the central event-processing system. It does not assume that physical destruction of the storage device can be recovered without additional replication or backup mechanisms.

---

6. Contextual Event Retrieval

The storage organization is designed around the main dimensions that are useful when investigating failures:

Who, When, and What.

The retrieval interface can be represented as:

```
retrieve(
   source | all,
   start_time,
   end_time,
   event_type?
)
```

A bounded time range is required for every retrieval operation. This prevents an investigation from accidentally requesting an unbounded amount of historical data.

For a query such as:

```
retrieve(
   source = 474,
   start_time = 10:07:00,
   end_time   = 10:07:10
)
```

the retrieval process can operate hierarchically:

```
Time Range
   ↓
Relevant Segment(s)
   ↓
Source
   ↓
Candidate Blocks
   ↓
Block Metadata
   ↓
Event Scan
   ↓
Exact Event Filtering
```

Block metadata contains information such as the source, temporal range, physical offset, and amount of valid data. This allows the system to eliminate blocks that cannot contain relevant events before scanning individual records.

The system does not need to perform an expensive direct lookup for every individual event. Instead, the goal is to narrow a potentially very large event history to a bounded region and then perform a sequential scan over the relevant blocks.

This becomes particularly useful for multi-agent failures. A query concerning one drone can request events from all sources within a bounded period when the cause may involve another agent.

---

7. Deterministic Event Decoding

The retrieved events are still binary data and cannot simply be provided to an AI model with the expectation that the model will correctly determine what every byte represents.

Therefore, the system separates **binary decoding** from **AI reasoning**.

The structure of supported payloads is provided through configuration or an appropriate schema. A deterministic decoder uses this information to convert raw bytes into structured values.

For example:

Event Type: Sensor Status

Bytes 0–1 → sensor_id
Bytes 2–5 → temperature
Byte 6 → status flags

The actual conversion is performed by deterministic software rather than by the LLM. This prevents the model from making assumptions about byte offsets, integer representations, floating-point formats, endianness, or other binary details.

If an event contains a payload for which no schema is available, the system does not attempt to guess its structure. The original bytes can remain preserved in storage, but they are not treated as semantically interpreted evidence.

This provides an important distinction between **unknown data** and **known data**.

For example:

Known schema
↓
Decode
↓
Structured evidence
↓
AI reasoning

Unknown schema
↓
Preserve raw data
↓
Mark as unconfigured / uninterpreted
↓
Do not invent an interpretation

The dashboard can explicitly inform the user when relevant event data could not be interpreted.

---

8. Domain Knowledge and RAG

Decoding binary fields is different from understanding what those fields mean in the context of a particular autonomous system.

For example, a deterministic decoder may produce:

error_code = 17
planner_state = 4
sensor_status = degraded

Domain documentation may establish that:

error_code 17 → LiDAR timeout
planner_state 4 → avoidance maneuver

The system can provide such domain knowledge to the retrieval-augmented generation (RAG) layer. Domain knowledge may include system documentation, error descriptions, operational manuals, state definitions, or other relevant technical information.

The RAG layer retrieves the relevant portions of this knowledge along with the event evidence selected from the event store.

The AI model therefore receives information that has already passed through deterministic retrieval and decoding rather than directly accessing arbitrary raw storage.

If the available documentation does not establish the meaning of a particular value, the system should preserve that uncertainty rather than presenting an invented interpretation as fact.

---

9. Failure Synthesis

The final stage uses the retrieved evidence and domain knowledge to construct an explanation of the failure.

For example, an investigation may produce evidence such as:

Drone 298
10:07:02.8 → LiDAR degraded
10:07:02.9 → LiDAR timeout

Drone 474
10:07:01.2 → formation active
10:07:03.1 → avoidance triggered
Combined with the relevant domain knowledge, the reasoning layer may conclude that the sensor failure on Drone 298 likely preceded the avoidance behavior observed by Drone 474.

The important point is that the LLM is responsible for **correlating and synthesizing evidence**, rather than defining what the underlying binary data means.

The system can also support progressive investigation. If the initial bounded context is insufficient, another bounded retrieval operation can be performed to obtain additional evidence. This allows an investigation to expand its context without giving the AI model unrestricted access to the complete event history.

---

10. Dashboard and User Interaction

The dashboard provides the interface through which an operator interacts with the system.

A typical workflow is:

User
↓
Dashboard
↓
Failure Query
↓
Relevant Event Retrieval
↓
Decoding + Domain Knowledge
↓
RAG / AI Reasoning
↓
Failure Synthesis
↓
Dashboard

The final dashboard output can contain:

- A synthesized explanation of the failure
- Relevant supporting events
- The sources involved in the reconstruction
- Relevant temporal information
- Limitations or missing evidence

For example, if an event contained an additional payload region with no configured structure, the system can explicitly show:

> **Uninterpreted data:** Event type `0x31` contained additional payload bytes whose structure is not configured. The bytes were preserved but were not used for semantic interpretation.
>
> This ensures that the system communicates not only what it believes happened, but also where the available evidence was incomplete.

---

11. Expected Outcome

The expected outcome of the project is a working prototype capable of receiving structured events from distributed sources, storing them in a crash-tolerant event store, efficiently retrieving bounded historical context, deterministically decoding supported event types, and using relevant domain knowledge with an AI reasoning layer to synthesize explanations of failures.

The project combines systems-level event processing and storage with retrieval and AI-based reasoning. Its central objective is to make failures that are otherwise difficult to reproduce or manually investigate more explainable by preserving the evidence surrounding them and reconstructing that evidence in a structured manner.

Rather than treating the AI model as the source of truth, the system establishes deterministic boundaries around data preservation, retrieval, and binary interpretation. The AI layer operates on validated evidence and documented domain knowledge, while unknown or unsupported information is explicitly identified to the user.

In this way, the **Failure Synthesis System** aims to provide a practical framework for reconstructing and explaining failures in autonomous and distributed systems from historical event evidence.